

William Cannella

Advanced Machine Learning

3/16/2026

Capstone Project Milestone

This project involves using machine learning models to predict stock returns based on congressional trading patterns disclosed under the STOCK act (2012). Since beginning this project significant progress has been made, moving from just a conceptual research question to an automated pipeline for data collection, feature engineering and baseline modeling. The implementation now includes 9000+ trades with hundreds of different members of congress and hundreds of different stocks with leadership positions and weighted committee power scores, and temporal features for each trade. Preliminary results demonstrate that ML models can capture signals and congressional trade metadata, with the Random Forest model achieving 70.3% accuracy and a ROC-AUC of .695 in identifying grades which outperform the S&P 500. Early feature importance reveals that reporting lag, trade amount and member age are the strongest predictors of a successful trade. The project repository (<https://github.com/wdcannella/congressional-trades-ml>) contains operational code with a web-scraper, robust data processing pipelines, and jupyter notebooks documenting analyses.

To give some background context, members of congress have privileged access to classified national security briefings, advanced knowledge of regulatory changes, early awareness of government contracts, and insight into upcoming legislation which has historically led to above average returns on trades and an abundance of wealth among congress. The STOCK Act mandates 45-day disclosure of trades, which was meant to inhibit congress from insider trading, but many investors agree it has not. This project aims to answer the primary research question: “Can machine learning models predict whether a congressional stock trade will outperform the S&P 500 index?” as well as secondary questions such as “Which member characteristics predict trading success?”, “Are there party-based differences in trading outcomes”, and others.

The project works from 3 primary data sources through an automated pipeline. These include Capitol Trades Website, Yahoo Finance API, and Congressional Records. Capitol Trades data is collected using a selenium based web scraper which navigates to the home page, collects data, and sifts through hundreds of pages. It has an incredible success rate, and collects data with a 97% match rate with congressional records. It takes about 10 seconds per page collecting 96 data points per page, with 361 total pages. It outputs the following columns per trade: politician_name, party, chamber, state, ticker, issuer_name, traded_date, published_date, transaction_type, size, price, owner, filed_after_days, scraped_at. The congressional metadata and committee membership is sourced from <https://github.com/unitedstates/congress-legislators?tab=readme-ov-file>. All data is then sent through

src/process_data.py which compiles data for each trade, calculating feature scores such as committee power scores. It also fixes data discrepancies in features like names, so that metadata can be matched to trade data. For example a trade might have “Shelley Moore Capito” while the metadata will have “Shelley Moore”. These differences are sorted through and matched to the best of my ability in this file. This data is then sent through download_stock_prices.py and calculate_returns.py where specific stocks are downloaded to match those found in the congressional trades, and returns are calculated for each stock. Returns are calculated at 30, 60, and 90 day periods and compared to S&P 500 returns for that time period. Some issues I’ve faced with this portion of the dataflow is delisted stocks and weekend trading leading to NaN values.

Some detailed feature engineering has been done to capture political influence and how it affects trading. One very important feature is the Committee Power Score. This score quantifies a member’s legislative influence. The score is calculated based on the below table according to the economic impact of committee jurisdiction, access to non-public information, and regulatory authority over industries. These scores will likely be adjusted in the future as the models are tweaked.

Committee ID	Committee Name	Weight
SSFI	Senate Finance	5
SSAP	Senate Appropriations	5
SLIN	Senate Intelligence	5
HSWM	House Ways and Means	5
HSAP	House Appropriations	5
HSIF	House Energy and Commerce	4
SSBA	Senate Banking	4
SSFR	Senate Foreign Relations	4
Other	Standard Committees	1

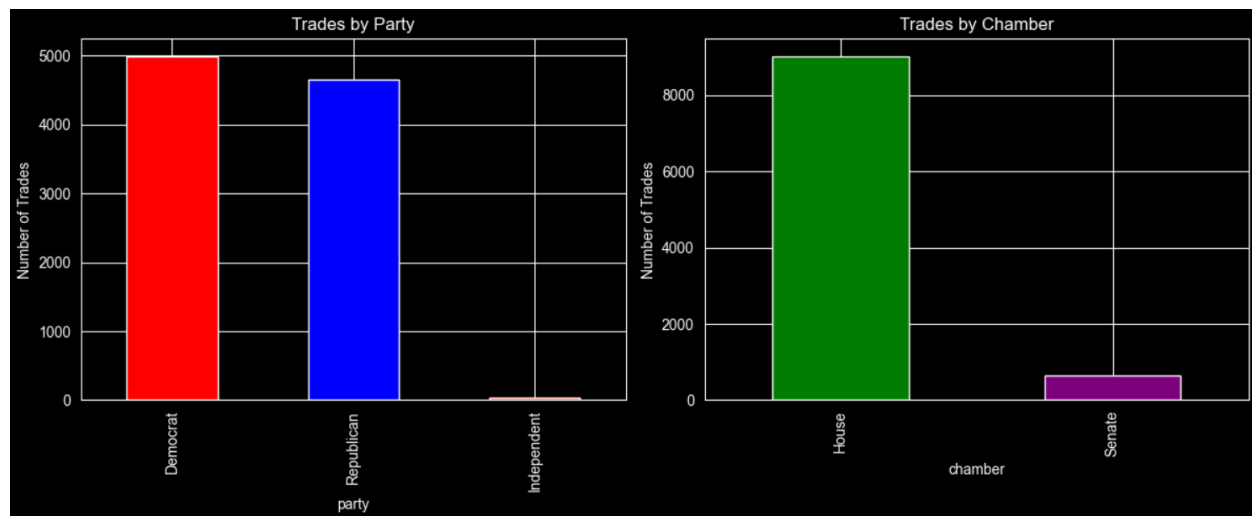
Another important feature is trade amount. The disclosure reports provide trade sizes in ranges (“\$15,001-\$50,000”). To convert this to a more continuous number, we calculate the midpoint of each range. For example, “\$1K-\$15K” is mapped to \$8,000. This allows ML models to treat transaction size as a quantitative variable. Another huge feature is the Leadership flag. This is a binary feature to identify party officials (Pelosi, Schumer) who have broad legislative oversight in the following positions:

Senate: Majority Leader, Minority Leader, Whips

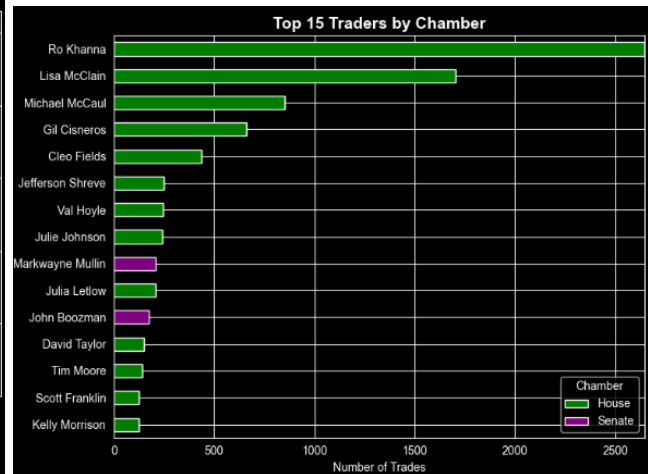
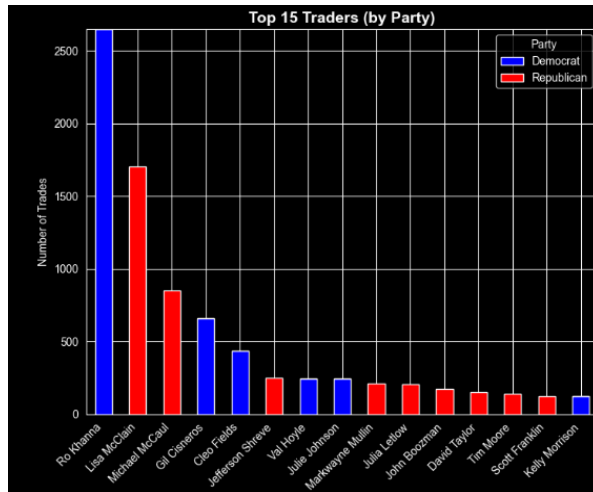
House: Speaker, Majority Leader, Minority Leader, Whips

The last important measure is the Filed After Days feature. This measures the reporting lag between the date the trade occurred and the date the trade was disclosed to the public. This feature is likely important because a major delay in reporting might miss the profitable window the congress member used.

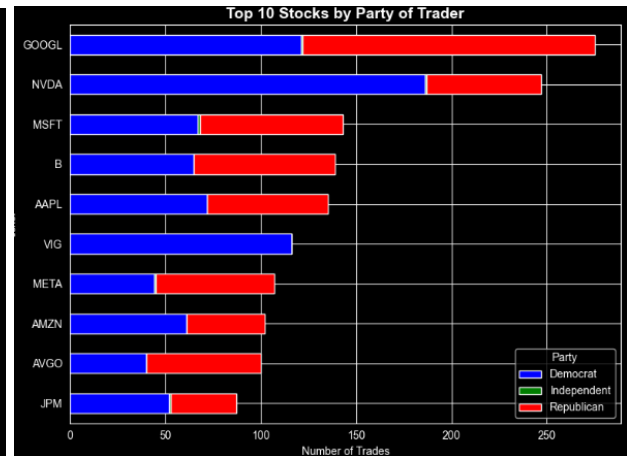
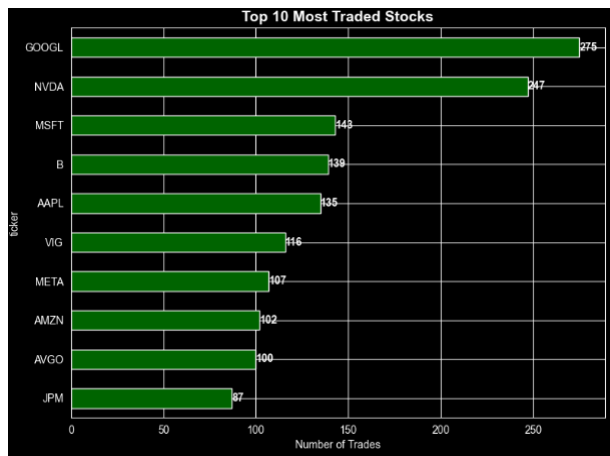
Some exploratory Data Analysis has been completed, identifying key patterns and challenges. When it comes to party and chamber, results are largely as expected. With 48.1% of trades being from republican members of congress, and 51.6% being from democrat members. Along with that about ~9000 trades occur from the House while only ~500 come from the senate. While there are many more members of the House this still shows more trades per member of the house compared to the senate.



When looking at the most active traders there is some interesting information being displayed in the two figures below. While the by far most active trader Ro Khanna is a democrat, only 6/15 of the top traders are democrats and the remaining 9 are republican. Despite there being more democrat trades overall, this could show that republicans in general place larger amounts of money on their trades compared to democrats. Along with this 13/15 of the top traders are a part of the house of representatives. This is likely due to the bias of having so many more members of the house compared to the senate.

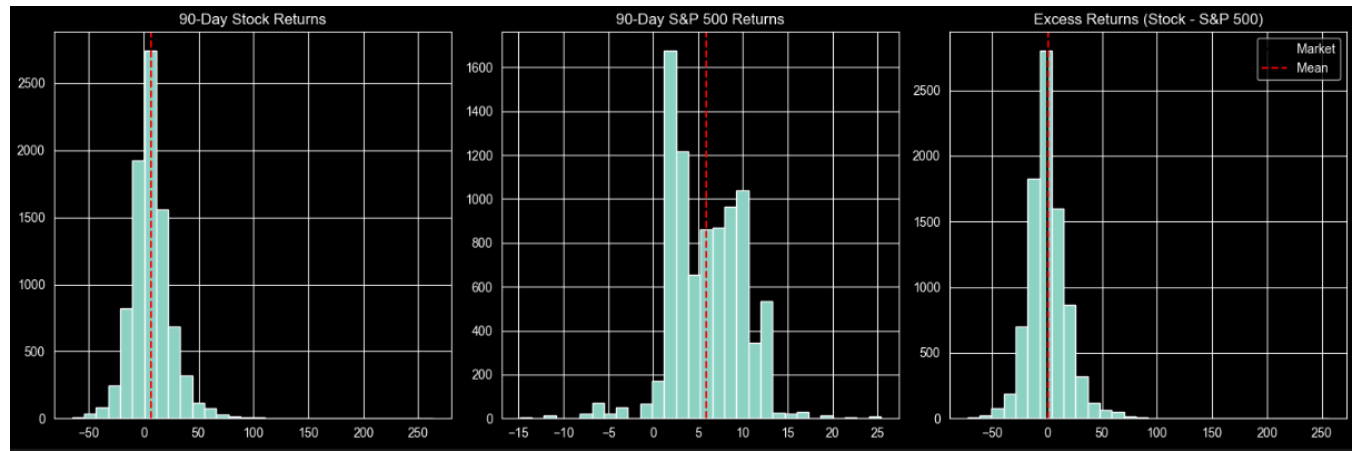


The most traded stocks are Google, Nvidia, Microsoft, Barrick Mining, Apple, Vanguard, META, Amazon, Broadcom, and JP Morgan with Google and Nvidia being the clear leaders by over 100 trades. All of these companies are either in technology or banking. Not much is said by analyzing the top stocks by party, with 9 being split almost evenly by democrats and republicans and the only outlier being Vanguard which is only traded by Democrats in recent years.



Moving on to the trade returns, the mean stock return of trades after 90 days was 6.55% compared to the mean S&P return of 5.90%. This creates a mean excess return of .65%, with only 41.3% of trades outperforming the market. This is an interesting metric because it shows that while congress is generally successful in their whole portfolio, individual stocks are more often not as successful as the market. Along with this we saw that republicans did better in their ventures, beating the market by 1.13% compared to the democrats who only beat the market by .18%. A very compelling feature is the performance by chamber where the Senate performed much better than the house, 4.07% compared to .47%. I also saw how filing delays affected the stocks performance with delays between 15 and 30 days being the highest performing at 2.09% above the market and 30 to 45 days was 1.72% below the market.

This goes against my initial thought that larger delays would perform better because of members trying to keep their trades under the radar. All of these metrics are playing a huge role in our models and predictions as they can very clearly help to predict a stock's performance.



More analysis can be found in notebooks/exploratory_analysis.ipynb, and this will continue to be expanded as the project progresses. Some of the biggest challenges when processing data was politician naming and date normalization. Politician's names often vary between data sources, so a congress member's name could be different on the trade report compared to their name on the committee membership dataframe. To combat this I developed a fuzzy matching logic to make matching easier between dataframes. I also addressed specific formatting issues like non-standard month abbreviations ("sept" vs "sep") to ensure consistency throughout the data.

The two models that I've implemented at this point in the project are Logistic Regression and Random Forest. To implement Logistic Regression I used the `sklearn.linear_model.LogisticRegression` implementation. As a baseline I set `random_state` to 42 and `max_iter` to 1000. This model will calculate the probability of a trade outperforming the market as a sigmoid function of the weighted sum of the input features. The large `max_iter` ensures convergence for the encoded features. For random forest I also used `sklearn.ensemble.RandomForestClassifier` and set `n_estimators` to 100, and `random_state` to 42. I used this model as a non-linear baseline. This implementation will create 100 trees on random subsets of the data. This should avoid skewing towards outliers, and integrate interactions between features naturally. This also simplifies the feature important score calculation.

Some preliminary experiments were conducted using the Logistic Regression and Random Forest models. These simplified results can be found below. The Random Forest Model significantly outperforms Logistic Regression, particularly in its ability to identify outperforming trades (Recall: 26.8% vs .3%). The key influential features in the Random Forest model include `filed_after_days` (reporting lag), `trade_amount`, and `age`.

Model	Accuracy	Precision (Class 1)	Recall (Class 1)	ROC-AUC
Logistic Regression	66.1%	40.0%	0.3%	0.521
Random Forest	70.3%	64.3%	26.8%	0.695

These results will be largely expanded upon in the next portion of the project. Some improvements that I plan on implementing include extending feature engineering to produce better results. All feature calculations should be honed to maximize results, and new features could be added. Data analytics will be vastly improved upon so that I can much more clearly see how the different features can relate to each other so models can be honed. This would have been done in this milestone but I was not able to due to time constraints. Along with this, more specific result analytics must be done in order to better hone the models. If enough time is available I will also implement more advanced models.

Bibliography

[1]unitedstates, “GitHub - unitedstates/congress-legislators: Members of the United States Congress, 1789-Present, in YAML/JSON/CSV, as well as committees, presidents, and vice presidents.” GitHub, 2025. <https://github.com/unitedstates/congress-legislators?tab=readme-ov-file> (accessed Mar. 18, 2026).